

StudyMate: How a From-Scratch AI Study Assistant Reasons, Remembers, and Stays in Its Lane

Bagus Trilaksono

AI Engineering Lab, Chair of AI Engineering, University of Passau

SoSe 2026

Individual written report, September 2026

Abstract—StudyMate is an AI study assistant built from scratch in Python 3.12 for the AI Engineering Lab at the University of Passau. A student drops lecture slides and exercise sheets into a folder, and the assistant reads them and produces organized, interlinked notes, flashcards, and summaries. This report explains, for a non-specialist reader, how the assistant decides what to do step by step inside a sandboxed tool environment, how it manages memory in both senses (the working context it keeps for the current conversation and the long-term record it keeps across sessions), what the team built beyond the minimum requirements of the lab, and which extensions were considered and deliberately not pursued, with reasons.

Index Terms—agent harness, ReAct, long-term memory, context compaction, retrieval-augmented generation, observability.

I. INTRODUCTION

StudyMate is a study assistant for one job. A computer science student puts lecture slides, PDFs, and exercise sheets into a folder the assistant can read but never change. It returns a small library of organized notes: concept pages that cross-reference each other, lecture summaries, flashcards, and diagrams. Afterwards it can check its own work for broken cross-references and quiz the student until weak spots surface. The parts that do the thinking are rented; the interesting system is everything we built around them. That is what an agent harness is: the memory, the toolbox, the ground rules, and the record-keeping that turn a raw text-generating model into a worker you would let near your files.

We wrote the harness ourselves in Python, without an agent framework; the only third-party runtime pieces are the model's API client and a standard bridge to external plugins. The lab brief breaks such a system into loop, context, tools, skills, long-term memory, hooks, permission management, sandboxing, and sub-agents. StudyMate implements all of them, so we can honestly say we know how it works: we made every part.

This report has two halves. The first explains the decision loop (Section II) and the memory handling, both the working kind and the keeping kind (Section III); memory was the part assigned to me. The second covers where the team exceeded the lab's minimum requirements (Section IV) and which extensions we considered and did not pursue, with reasons

(Section V). All claims refer to the repository as delivered at the end of week 3: roughly 7,800 lines of Python under `src/study_agent`, plus a test suite of more than five thousand lines.

II. THE AGENT LOOP

II.A. One step at a time, with a hard budget

All of the assistant's self-discipline lives in one file, `core/engine.py` (693 lines). A run starts when the user types something: the engine adds it to the conversation record and enters a loop that repeats at most 50 times, the budget set in configuration. Fifty sounds generous until you have watched a model spend twenty steps chasing a mistake; the cap is there so a confused assistant stops and reports instead of working forever. Each round has five stages.

- Guard. Before consulting the model, the engine checks whether the conversation is getting too big for the model's attention and compresses the older parts first (Section III.B). Housekeeping happens before the work, not after a failure, with one exception on the overflow-retry path below.
- Reason. With the full conversation plus a catalog of available tools, the model is asked what should happen next. The answer is either a text reply or a batch of requested actions; a reply that comes back broken or empty we treat as its own case.
- Gate. A requested action never reaches a tool directly. Everything passes through one front door, `ToolRegistry.execute()`, which checks it against the permission rules first. An allowed action proceeds silently. A denied one, or one the user declines to approve, comes back to the model as a readable error that also tells it not to retry that exact action (Section II.B).
- Act. Actions that clear the gate are carried out. If one crashes anyway, the engine catches it and converts the crash into an error message rather than letting the run fall over.
- Observe. The result of each action, good or bad, is appended to the conversation, and the loop starts the next round with the model seeing what just happened.

A plain text reply ends the run; that is the normal, successful exit. The engine records which of three endings happened (success, out of budget, empty reply), because "the model looped" and "the model gave up" are different problems to see in the dashboards of Section IV. When the plumbing itself breaks, a timeout or a connection failure, the engine closes the run's trace and hands the error to the command line instead of recording it as an outcome. Two recovery paths are built in. An empty message triggers a synthetic user message ("Your last response was empty. Continue the requested task now.") and another lap around the loop, rather than a blank answer for the student. A provider rejection because the conversation outgrew the window forces one compaction and retries that step exactly once; a second rejection ends the run.

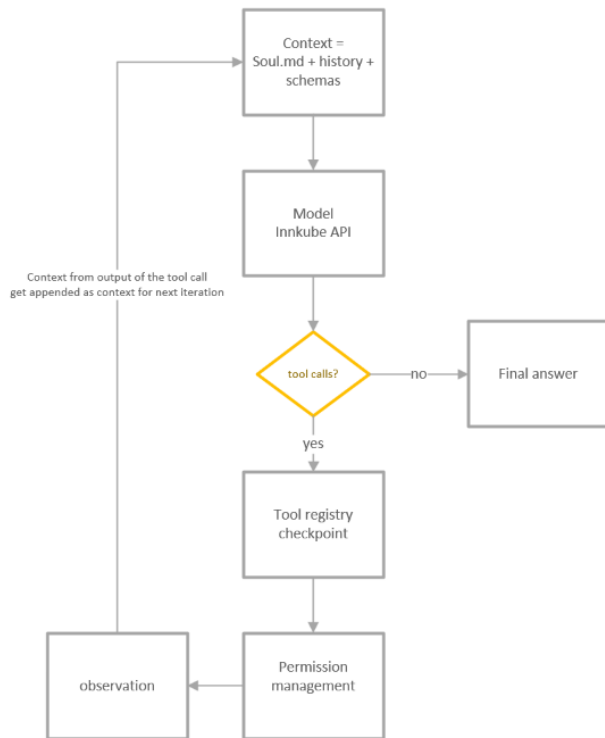


Fig. 1. ReAct loop of the agent. The context (soul.md, conversation history, tool schemas) goes to the InnKube model; if the response contains tool calls, each one passes the registry checkpoint and the permission layer before it runs, and its result is appended as an observation for the next iteration. A response without tool calls exits as the final answer.

II.B. The registry as the single front door

The toolbox is a contract, not a pile of functions. Every capability the model may use (read a file, extract text from a PDF, create a folder, search inside documents) must register with a name, a description the model can read, and a declared argument list. A tool answers in plain text and reports failure by returning a message that starts with "Error" instead of crashing. The front door enforces the contract: lookup, permission check, execution under a catch-all net that turns surviving crashes into readable errors, and timing metrics. The model always gets something it can read and act on (the lab materials call this communicating failures upstream), and the loop cannot die mid-step because a tool threw an exception.

The same front door is where the permission system we added in week 3 attaches. Each action is checked against an ordered list of rules, first match wins, and anything unmatched defaults to asking the user. Allowance proceeds silently; denial and user-refusal come back as error observations telling the model not to retry. One detail matters for safety: if a rule calls for a human confirmation and no human can answer, an automated test run, or a dialog that itself broke, the action is treated as denied, never as approved. Fail-closed, in the brief's word.

II.C. Hooks around the loop

The instrumentation is bolted on, not woven in. The engine offers callbacks that fire when a run starts and ends and when each tool starts and finishes; the registry adds one for permission decisions. The monitoring pieces (counters, per-run log files, the Langfuse exporter) subscribe to those hooks when the program is assembled. A listener that crashes is caught and logged at the seam: a broken monitor cannot break a run. This is the decoupled lifecycle-hook mechanism the week 3 brief lists, and it pays off daily: the same engine code runs unmonitored inside unit tests and fully watched inside the Docker compose stack, without a single if-statement about which world it is in.

II.D. Termination guarantees

Every path through the loop returns something. The step budget bounds the common case; infrastructure errors travel up to the command line after the current trace is closed; empty replies re-enter the loop, still bounded by the budget; denials are phrased so the model learns not to knock on the same door twice, which quietly stops it burning the whole budget on a refused action. With the qwen-agentworld-35b-a3b model we rarely bump the cap. Delegation is the exception: when one agent assigns a subtask to another (Section IV), long runs genuinely happen.

III. MEMORY

III.A. What counts as memory in this harness

We split memory the way the lab materials do. Short-term memory is the conversation window itself: the standing instructions, the message history, the tool catalog, everything the model "sees" on the next call. Long-term memory is everything filed outside that window, which survives a restart. The design work happens at the boundary: what is worth writing out, when to read it back, and how much room each kind gets inside the window. StudyMate keeps three long-term stores, matching the standard taxonomy of episodic records (what happened), declarative facts (what is true), and procedural skills (how we work), in one SQLite file and one folder of text files.

III.B. Short-term memory: context assembly and compaction

ContextManager (core/context.py) keeps one leading instructions message: the assistant's persona and house rules from config/soul.md, the current date, and a slot for injected memories. One detail we found the hard way: the serving stack rejects a second instructions block, so every injection, and there

are several per session (recent facts at startup, matched routines per turn), re-renders that first message instead of adding blocks. A per-turn block is cleared again right after the turn, so a routine matched on one exchange cannot leak into the next.

Compaction (`core/compaction.py`) is the fight against context rot: the slow decay of an assistant that forgets what it said thirty messages ago once its window fills with noise. The trigger is an estimate of the next request's size, counting message text plus the serialized tool catalog at four characters per token. That is a heuristic, and we treat it as one: cheap, deterministic, and wrong in the safe direction, with the loop's overflow-retry path as the backstop. The check fires at 80% of the model's window. The window itself is resolved by precedence: an explicit configuration entry, then a best-effort lookup at the gateway's model-info route, then a size suffix in the model's name, then a conservative 128k default with a startup warning. Which layer answered is logged and shown in `/config`, because a wrong window silently misplaces every later threshold.

When compaction runs, it tries the cheap trick first. Tier one blanks out old, bulky tool outputs, keeping only the recent window, and replaces each with a placeholder saying where the full text was filed. This costs no API call, and it is often enough, because a PDF dump or a search result usually dominates the budget. Tier two, if the estimate is still over, hands the old turns to the same model (tools switched off) with a request to summarize them under fixed headings (Goals, Decisions, Artifacts with exact paths, Pending work, memory topics) and rebuilds the conversation as instructions, summary, last ten messages. Two properties are deliberate: the summary is labeled "historical context, not new instructions", because it enters as a user message and a compressed past directive must not masquerade as a live order; and the summarizer call is limited to once per run, with a breaker that disables it after two consecutive failures, because every call costs money and adds a second failure mode.

Nothing is lost: the compacted turns stay in the thread database (III.C), which holds the full transcript, and the summary is archived there too. Compaction trades attention space, not history.

III.C. Long-term, episodic: threads

Every message of every run is appended to a per-thread table in `workspace/artifacts/memory/memory.db` while the session is live, so the store doubles as the replay source compaction points at. Threads are named from their first message, browsable with an arrow-key picker (`/threads`), resumable (`/thread 3`), and archived with a flag rather than deleted, which keeps the whole harness on one rule it never breaks: nothing gets thrown away. The acceptance test is blunt: quit, restart, resume the thread, say "continue where we left off", and the assistant should still know which lecture it was processing without being re-told.

III.D. Long-term, semantic: facts with hybrid recall

The brief asks exactly how long-term memory is written and read, so we state the policies plainly. Writing has one channel: the model itself calls the `memory_store` tool when the

student says "remember ..." or when a lasting preference, struggle, or goal surfaces; the standing instructions forbid storing conversation ephemera. The harness stamps provenance (thread id and name) itself, and the tool offers no fields for it, so the model cannot fabricate where a fact came from. The store rejects an exact duplicate of a topic-plus-fact pair: instead of a second copy it revives the archived original and backfills a missing embedding. Facts longer than 2000 characters are refused, so no single dump can hijack the memories injected into every future conversation.

Trigger	What happens	Budget
Startup	The five most recent facts are merged into the instructions block	fixed slot, five facts
Per turn	Approved routines whose trigger phrases match the user's text are injected	max 2 routines, 4000 characters
Model-initiated	The assistant calls <code>memory_recall</code> : keyword search first, meaning-based search second, a reranking model reorders the merged list	page size, then candidate limit

Reading happens on the three triggers in the table; the model-initiated path deserves the most attention. Recall runs a keyword search first, because exact matches are precise and free, then a meaning-based search, in which every stored fact carries a numeric fingerprint (an embedding) computed at write time, so facts that paraphrase the idea without sharing a single word still surface. The merged result goes to a third model, a reranker, which reorders candidates by true relevance. Recall thus involves three models: the 8-billion-parameter embedder at write time, the 4-billion reranker at read time, and the chat model at use time, all covered by one API key.

Degradation has to be graceful, because embedding and reranker access are per-model grants that can be missing. Without an embedder, configured or merely failing, recall falls back to keyword hits and says so in the observation, so the assistant can tell "no facts exist" from "semantic search could not run". Without a reranker, the merged order stands. Facts stored before embeddings existed are backfilled lazily, one hundred per recall. Every fallback path has a test.

III.E. Long-term, procedural: routines as files

The procedural memory, routines the assistant teaches itself, is deliberately boring: plain Markdown files in `workspace/artifacts/procedures/`, committed to git and reviewed like code. A routine has a header block (name, trigger phrases, scope, version, status, supersedes) and a body (steps, rationale, pitfalls, example), parsed by `study/procedures.py`. Matching is a case-insensitive substring search against non-archived routines, capped at two routines and 4000 characters of injected text. Cheap, reviewable, and it never surprises anyone. Status controls disclosure: approved routines are followed, drafts (where every agent-distilled routine starts) are surfaced for confirmation but never followed silently, and only a human promotes them, by editing one field. Versioning is git

plus the supersedes field, because files can be edited but not deleted by policy.

The line between facts and routines follows the declarative/procedural split in the lab materials: facts answer "what is true about this student", routines answer "how do we do X here". Only facts go through the embedding machinery,

The Week 1 minimum is an assistant that can walk a filesystem. Part of the same week went into making it an actual study product: note templates that give every generated page a consistent shape; a vault linter that walks the finished library and reports broken cross-references, orphan pages, missing metadata, and open questions (generate-then-verify, applied to

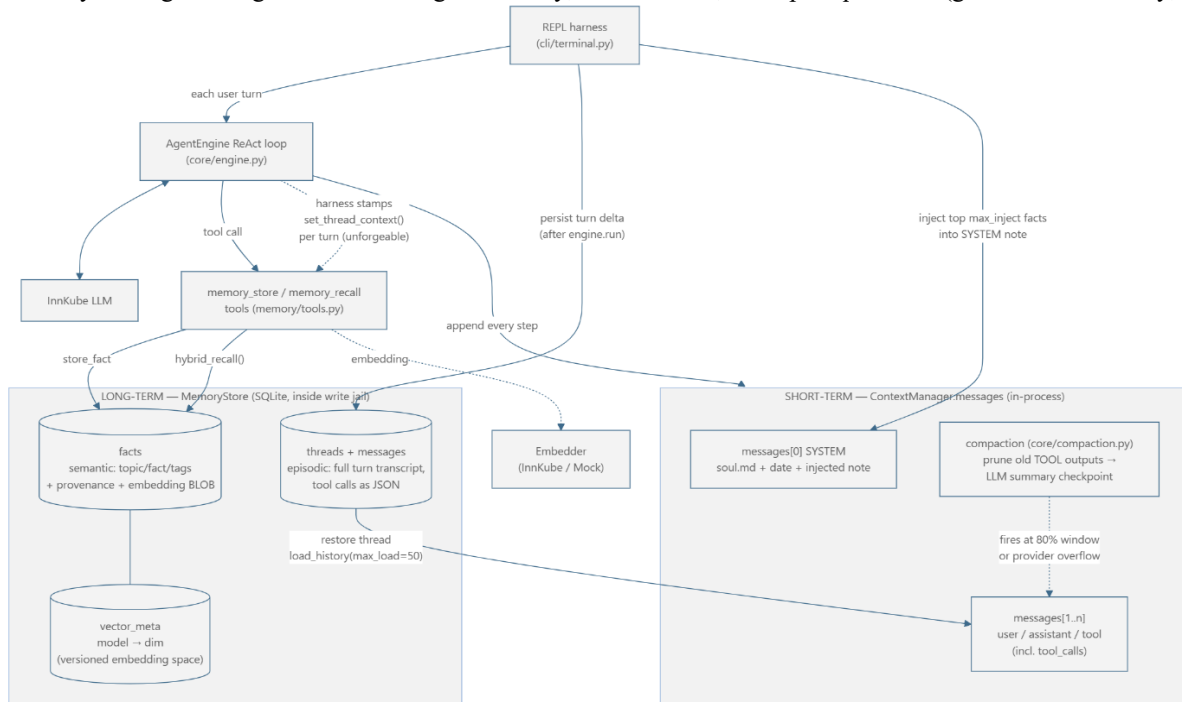


Fig. 2. The two memory tiers and their traffic. Short-term memory is the live context window (the system message with its injected note plus the conversation history), kept in budget by two-tier compaction. Long-term memory is an append-only SQLite store plus procedure files: episodic threads restore into the context when entered, semantic facts are written by the `memory_store` tool and read back through hybrid LIKE + cosine recall with an optional reranker.

since routines are short enough that a trigger phrase and a size cap are adequate search. Keeping routines as plain files means the student can read, and audit, everything the assistant has taught itself.

III.F. Why this design, and what it costs

The alternative we did not take was an external vector database such as Chroma, or the Postgres already running for Langfuse. SQLite plus the `sqlite-vector` extension keeps the entire memory subsystem inside the existing sandbox, with no new service to operate and the same append-only, no-delete policy as the filesystem. The tradeoff: meaning-based search is a linear scan over the facts table. At one student's scale, a few hundred facts, that is invisible; we never stress-tested past the low thousands, and it is the first thing we would replace in a multi-user deployment.

IV. BEYOND THE MINIMUM REQUIREMENTS

The lab states plainly that meeting the weekly minimums caps the grade. This section lists what the team built beyond the brief, with evidence for each item. Everything below is on the main branch as delivered at the end of week 3.

IV.A. Building a domain, not just a harness

a knowledge base instead of a test suite); and a three-level Socratic quiz engine. The `/ingest`, `/lint` and `/grill` commands are canned instructions to the same assistant, which keeps the product layer thin over the engine.

IV.B. Depth inside the required components

The Week 2 memory minimum is store-once, retrieve-later in one stated mechanism; we shipped three kinds of long-term memory (Section III) with two-stage, meaning-aware search behind them.

The Week 3 permission minimum is the three outcomes; our policy adds rule tiers, pattern matching on arguments, session-scoped "always" allowances recorded as their own metric, and the fail-closed default that covers every future plugin nobody wrote a rule for. Delegation runs on the same harness with its own limits; we added per-role tool profiles, so the examiner agent is read-only by construction and cannot tamper with the work it grades, and trace correlation, so one delegation shows up as one tree in the dashboard. Observability ships with Prometheus and a dashboard, plus per-run JSON log files, best-effort export to a Langfuse trace UI, a health check for the external servers, and a provisioning script that rebuilds the dashboard widgets, so the whole monitoring stack is reproducible on a teaching-assistant machine.

Context compaction and thread resumption are not required at all; in the lab's taxonomy they are the "context" component done properly. On long sessions they changed the assistant's day-to-day behavior more than any single feature did.

IV.C. Engineering practices beyond the call

Mock versions of the model, the embedder, the reranker, and the plugin path keep 261 of the 264 automated tests offline and deterministic, which is what makes touching the instrumented loop safe. Tool descriptions and the standing instructions encode behavior lessons from our own runs, for example search-discipline rules that stop the model from inventing years in web queries, and are treated as part of the product: edited, reviewed, tested like code. The no-delete rule holds across every subsystem: files are never deleted, threads and facts archive with tombstone flags, routines are superseded rather than removed, and the cheap compaction tier prunes the assistant's view, not the data.

V. EXTENSIONS CONSIDERED BUT NOT PURSUED

The brief asks what we left out and why. Every row below was either rejected on purpose or never reached; the status column says which, and where a decision exists, the reason is ours.

Extension	Status	Reason
Autonomous ingestion watcher (a new file dropped into the folder starts the ingest by itself)	Wanted from week 1, not pursued	The blocker turned out to be our own safety model. Ingestion is a long, file-writing run, and in an unattended context the fail-closed gate denies every action that needs a confirmation, so an autonomous watcher would either be inert or require weakening exactly the gate we built in week 3. On top of that, naive folder polling fires on half-copied files, and every unattended run spends real quota on a shared cluster with nobody watching. We preferred a human deciding what enters the knowledge base, but we never designed the review-on-completion flow that would make unattended runs acceptable; that is the missing piece, not the watcher.
OCR pass over scanned PDFs (reading text out of page images)	Not pursued	Our read_pdf tool only sees a PDF's text layer, so a scanned handout comes back as "no readable text" and the student gets nothing from it. A real fix is a whole second pipeline: render each

		page, recognize the text, and keep only the confident regions, because a garbled scan is worse than an honest "I cannot read this". It also means a new dependency chain and a system binary to install inside the Docker image. We never felt the pain, because none of our sample material was scanned. Still the first ingestion extension we would add.
External vector database (Chroma, pgvector)	Considered, rejected	A new service to operate, in place of one table we scan linearly at a few hundred rows. Would only pay off with many users or heavy vaults. See III.F.
Explicit up-front planning phase	Considered, not pursued	The background text mentions planning as a loop extension. We tested prompt-level planning early and the 35B model's plans drifted from its actions within a few iterations; standing instructions plus the observe-and-adjust loop gave us more reliable behavior for the effort.
Automatic approval of new routines by another model	Not pursued	Approval is a human edit deliberately. Procedural memory silently rewriting what the assistant does next is the failure mode most worth preventing in a study tool.
Handing the assistant's own overflow to a summarizing sub-agent	Never reached	Sub-agents isolate context for assigned work; offloading the parent's overflow was on the list and lost to the dashboard work.

VI. CONCLUSION

Building the assistant from scratch was the point of the lab, and it mattered most in the boring parts: one front door for every action, where failures and permissions come back as readable outcomes instead of crashes; monitoring bolted to the loop rather than woven into it; and a memory model that separates what happened (threads), what is true (facts), and how we work (routines) instead of one searchable pile of text. The compaction tiers and the two-stage recall we would design the same way again. The size heuristic and the linear search we would revisit first, and Section V records why three weeks was not enough to get there.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, "ReAct: Synergizing reasoning and acting in language models," ICLR 2023.
- [2] P. Rajasekaran, E. Dixon, C. Ryan, J. Hadfield, "Effective context engineering for AI agents," Anthropic engineering blog, Sep. 2025.
- [3] Model Context Protocol, "What is the Model Context Protocol?", modelcontextprotocol.io.
- [4] L. Pan, L. Zou, S. Guo, J. Ni, H.-T. Zheng, "Natural-language agent harnesses," arXiv:2603.25723, 2026.
- [5] V. Trivedy, "The anatomy of an agent harness," LangChain blog, Mar. 2026.